

MSDM 5001

Introduction to Computational and Modeling Tools

Lecture 2: Version control with git and GitHub

"FINAL".doc



FINAL.doc!



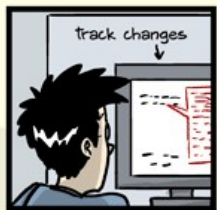
FINAL_rev.2.doc



FINAL_rev.6.COMMENTS.doc



FINAL_rev.8.comments5.
CORRECTIONS.doc



FINAL_rev.18.comments7.
corrections9.MORE.30.doc



FINAL_rev.22.comments49.
corrections.10.##\$%WHYDID
ICOMETOGRADSCHOOL?????.doc



JORGE CHAM © 2012

What we want:

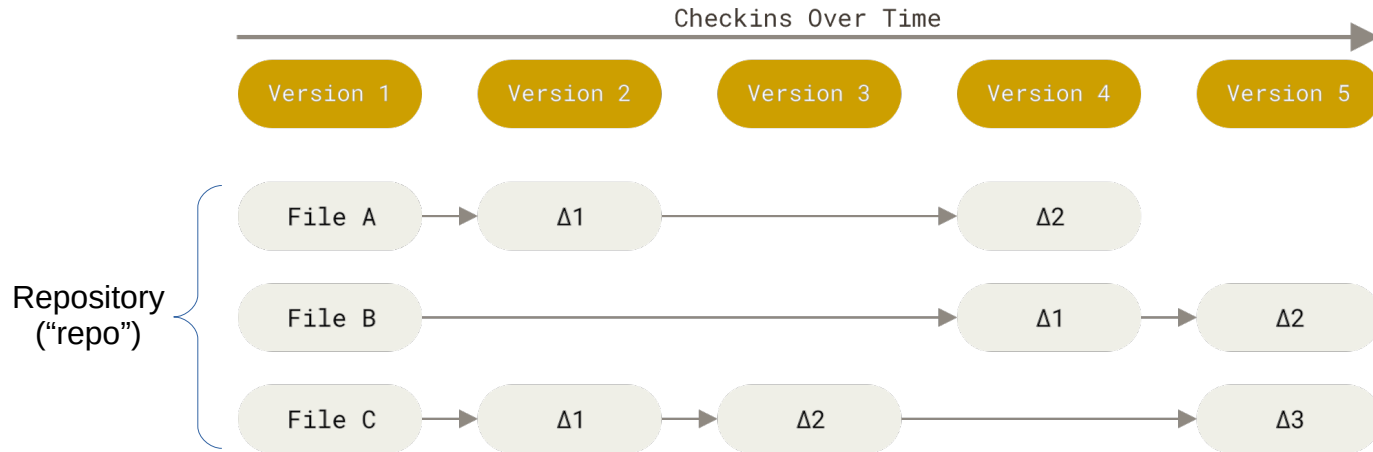
- Record changes to files over time
- Back up entire history of files
- Show what changes between versions
- Revert back to earlier versions if needed
- Enable collaboration with multiple people

One solution:  **git**



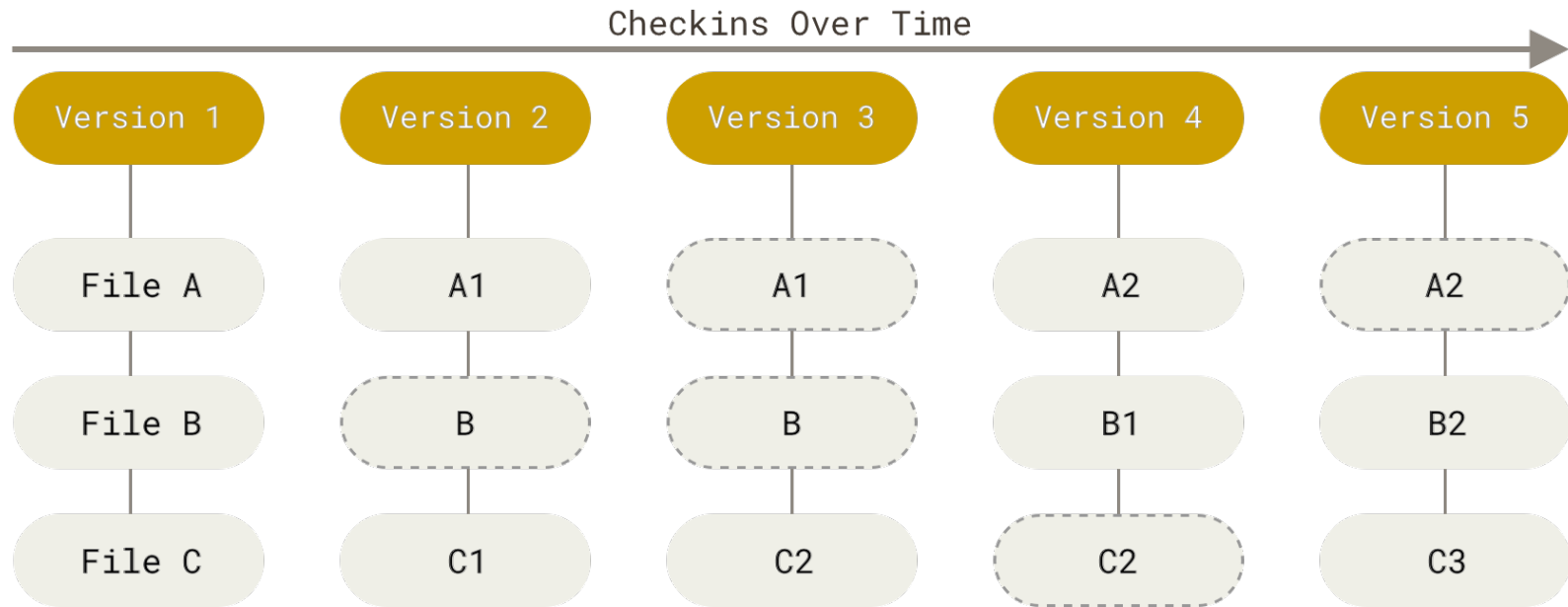
How would we build a version control system?

- Naively: copy files into another (timestamped) directory
 - Issues?
- More sophisticated: a database system
- One method: delta-based



Version control in Git

- Like a miniature filesystem with snapshots



Pros and cons?

Setting up Git

- Install: `sudo <package manager> install git`

```
$ git config --global user.name "Your Full Name"
$ git config --global user.email "your.email@example.com"
$ git config --global init.defaultBranch main
$ git config --global core.editor "nano" # or vim, code, etc.
```

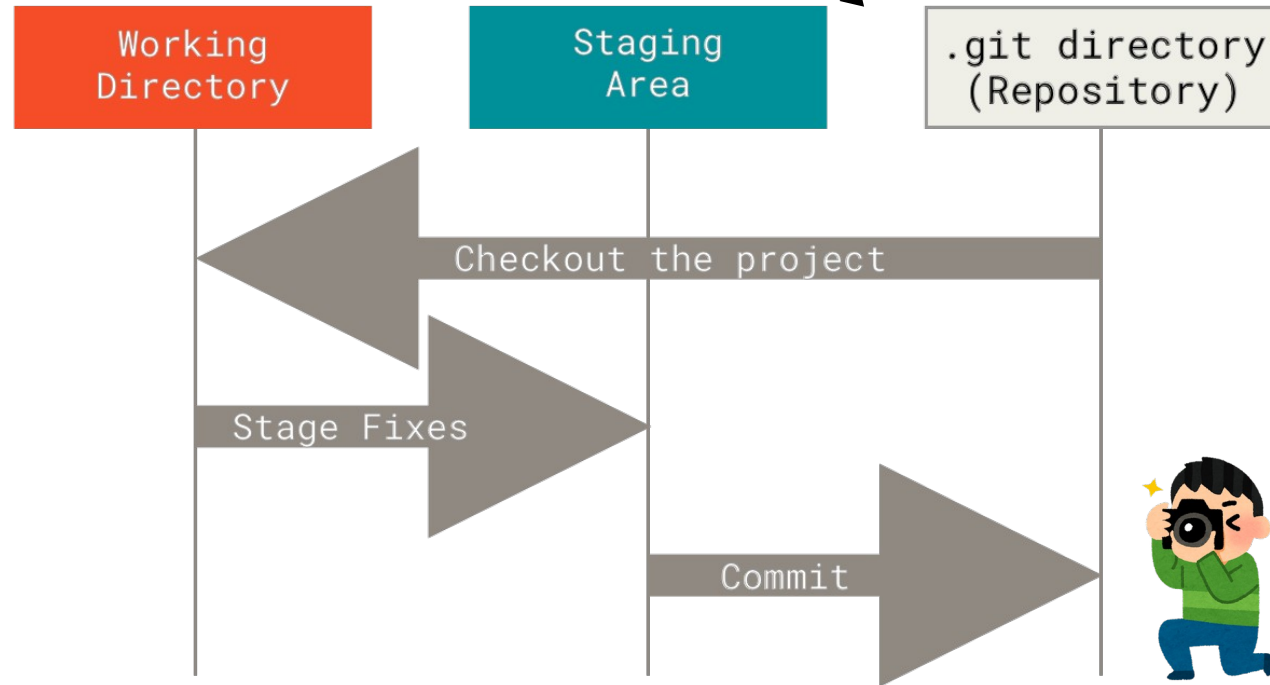
Check your configuration:

```
$ git config --list
$ git config user.name
$ git config user.email
```

The three states of files in Git

- Modified, staged, and committed

Why?



What is stored every commit

- Snapshot of all files (incl. sub-directories)
- Author & timestamp
- Commit message
- Reference to previous commit (tree structure)
- A hash identifier

What's inside .git?

- **objects**: actual file contents & metadata
- **refs**: pointers to commits
- **HEAD**: pointer to current commit (***)
- **Index**: the staging area
- **config**: repo-specific configurations

Starting a project with Git

- `git init`

Initialize a repo

- `git clone`

Download a repo and save it locally

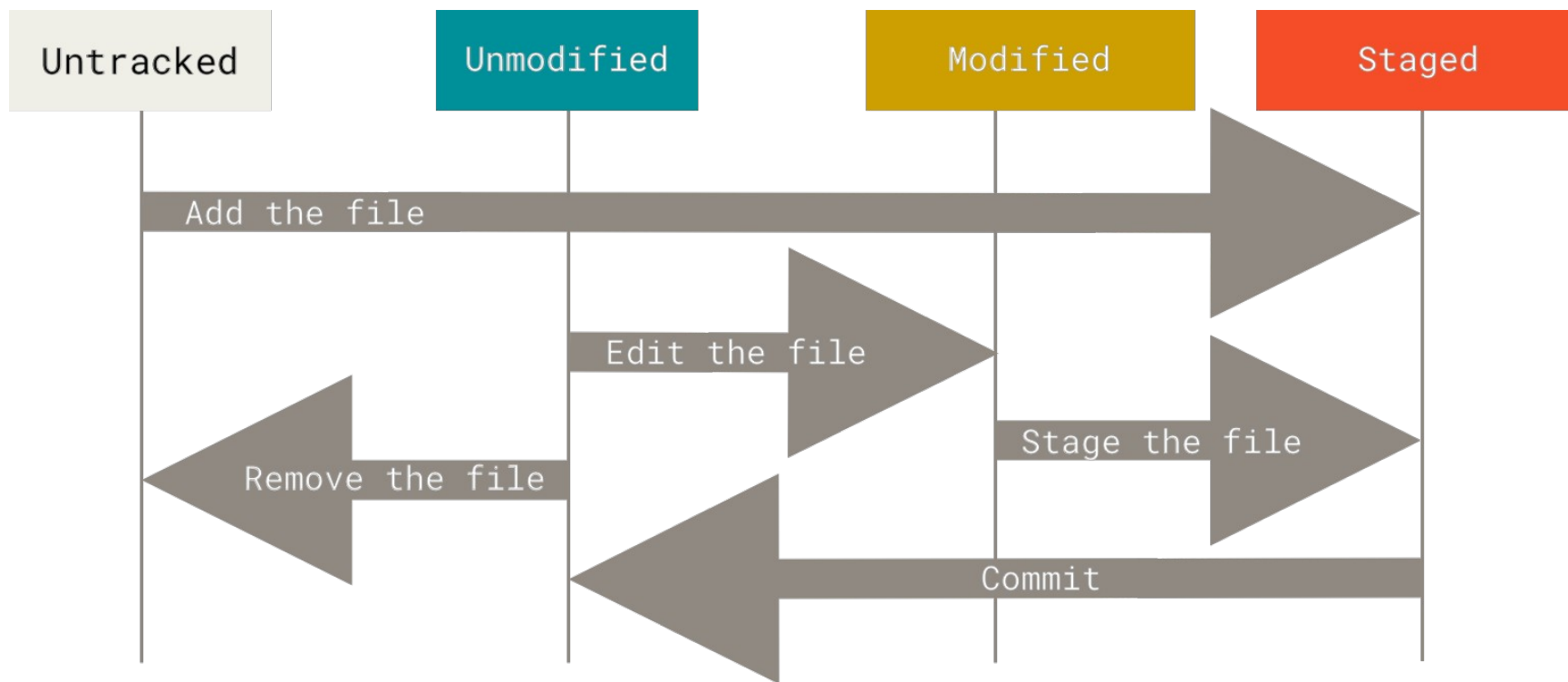
Option 1: Start a new project

```
$ mkdir my-project  
$ cd my-project  
$ git init  
Initialized empty Git repository in /path/to/my-project/.git/
```

Option 2: Clone an existing repository

```
$ git clone https://github.com/user/repository.git  
$ cd repository
```

Basic workflow



Basic workflow

```
1 # Check repository status
2 $ git status
3
4 # Add files to staging area
5 $ git add filename.py
6 $ git add .                # Add all files
7 $ git add *.py            # Add all Python files
8
9 # Commit staged changes
10 $ git commit -m "Add initial analysis script"
11
12 # View commit history
13 $ git log
14 $ git log --oneline        # Compact view
15 $ git log --graph          # Visual graph
```

git status

```
eric@fedora:~/Documents/demo_repo$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        analysis.py

no changes added to commit (use "git add" and/or "git commit -a")
eric@fedora:~/Documents/demo_repo$
```

git add .

```
eric@fedora:~/Documents/demo_repo$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
        new file:   analysis.py
```

git log

- Reverse chronological order
- Shows commit messages
- Shows commits' hashes

```
eric@fedora:~/Documents/demo_repo$ git log
commit 9d2f9f7a60bb91a6dd3bf934a1ee2ef264e7eea6 (HEAD -> master)
Merge: 51cb4bb a6537f8
Author: eric <syckeric@gmail>
Date: Tue Sep 2 21:47:41 2025 +0800

    Merge branch 'new_feature'

commit a6537f8d5d1240f03f7f0718a414709c14c03436
Author: eric <syckeric@gmail>
Date: Tue Sep 2 21:46:11 2025 +0800

    "Improved first heading text"

commit 51cb4bb419eead805746f270c66a876c968c933e
Author: eric <syckeric@gmail>
Date: Tue Sep 2 21:41:14 2025 +0800

    "Modified first heading"

commit 937b64909fbc5896b7d05aef035d81c923b529fd
Author: eric <syckeric@gmail>
Date: Tue Sep 2 21:22:57 2025 +0800

    "Added new feature"

commit 1c1966c99205abaa110ca25c45b3d2e8778f1e79
Author: eric <syckeric@gmail>
Date: Tue Sep 2 17:27:22 2025 +0800

    Added new heading

commit 27ca17b12e1e3dd5c89d13c94d1e43f94d642410
Author: eric <syckeric@gmail>
Date: Tue Sep 2 17:22:44 2025 +0800

    Added README.md
eric@fedora:~/Documents/demo_repo$
```

git diff

- Shows difference between working directory and staging area
(for tracked files only!)

```
eric@fedora:~/Documents/demo_repo$ echo "another line :)" >> README.md
eric@fedora:~/Documents/demo_repo$ git diff
diff --git a/README.md b/README.md
index d4f137b..4d1cd90 100644
--- a/README.md
+++ b/README.md
@@ -10,3 +10,4 @@ Some more text...

New feature!
A new line
+another line :)
eric@fedora:~/Documents/demo_repo$
```

`git checkout <hash>`

- Temporarily looks at an old commit (may branch off if desired, see later slides)
- Go back to latest commit with
`git checkout main/master`

Practice 1



1. Create a git repo called "git_practice" and cd into it
2. Check the status of your new repo
3. Create a new file titled "README.md"
4. Check repo status again, what do you see?
5. Make git track this new file, but don't commit yet. Check your repo status afterwards.
6. Commit with a clear message
7. View your commit history with git log
8. Make the following changes, and then check repo status.
 - i. Create a new file titled "notes.txt" with some texts to it
 - ii. Add some texts to "README.md"
9. Use `git diff`, what do you see? Does it make sense?
10. Commit again and view your repo's history graphically.
11. Use git checkout to look at an older commit. Notice how your working directory is changed. Finally go back to your latest commit.

Undoing in Git

- `git reset`
 - Syntax: `git reset [mode] [commit]`
- Affects entire repo!
- Three modes:
 - `--soft`
 - `--mixed` (default)
 - `--hard` ⚠



Undoing in Git

Modes	Working directory affected?	Index affect?	HEAD affected?
soft	✗	✗	✓
mixed	✗	✓	✓
hard	✓	✓	✓

* Use `git reflog` to undo undo

See demo

Undoing in Git

- Can also git reset specific files
 - Syntax:
`git reset [commit] <list of files>`
 - Changes the staging area ONLY!

Practice 2 (set up)

1. Make a new git repo "reset_practice". Create "main.py", and make a commit about it with the message "Initial commit"
2. Modify main.py to say `print("Version 1")` and create a new file feature.py with:

```
def feature_a():  
    return "Feature A"
```

3. Add both files and commit with message "Add feature A"
4. Modify main.py to say `print("Version 2")` and modify feature.py to:

```
def feature_a():  
    return "Feature A"  
  
def feature_b():  
    return "Feature B"
```

5. Add and commit with message "Add feature B"
6. Modify main.py to say `print("Version 3 – Debug")`, and create a new file "config.txt" with the line "debug=True"
7. Add and commit with message "Add debug configuration"
8. Check your repository status and log - you should have 4 commits total

Practice 2

1. You just made the debug commit, but realize you forgot to include some debug documentation. You want to undo the last commit but keep all your changes staged so you can add more files and recommit.

Afterwards, create a new text file "debug_notes.md" with text "Debug mode enabled".
Add and commit everything with message "Add debug configuration with documentation"

2. After reviewing, you decide the debug configuration shouldn't be included at all. You want to undo the last commit and unstage the changes, but keep the files in your working directory to possibly use later.

Check your log - verify you're back to 3 commits `git reset --soft HEAD~1`

Delete the debug-related files from your working directory. Don't make any commit yet! `git reset --mixed HEAD~1`

3. You realize that feature B introduced a bug. You want to completely remove all traces of the "Add feature B" commit and go back to the state right after "Add feature A". Use the appropriate reset mode to completely revert your directory to the "Add feature A" commit. `git reset --soft HEAD~1`

Check your log - you should only see 2 commits.

Look at the contents of "main.py" and "feature.py" - what versions do you see? `git reflog`

4. Oops! You realize you actually needed feature B after all, and you want to get back to where you were before the hard reset. What should you do?

Another way to undo in Git

- More modern and preferred: `git restore`
 - Never moves HEAD (commit history intact)
 - Safer
 - File-oriented (c.f. `git reset`)

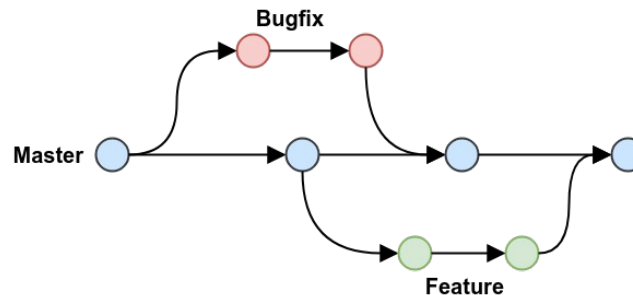
```
git restore file.txt           # Restore file from staging to working dir
git restore --staged file.txt  # Unstage file (restore staging from HEAD)
git restore --source=HEAD~2 file.txt # Restore file from 2 commits ago
```

git reset VS git restore

- Use `git restore` when:
 - Discarding changes in working directory
 - Unstaging files
 - Restoring specific files from commits
- Use `git reset` when:
 - Undoing commits (moving branch history)
 - Need to move HEAD/branch pointers
 - Resetting entire repository state

Branching

- Very often, you want to develop some features of your code independent from the main codebase
- Or you have a project with multiple collaborators
- Branches: independent lines of development
- Merged with master/main branch afterwards



Creating and switching branches

- `git switch`
 - switch to an existing branch
- `git switch -c:`
 - creates a new branch and switch to it
- `git switch -d:`
 - deletes a branch
- After switching, HEAD points to the branch, which itself points to a commit.
Working directory will be modified
- `git branch:`
 - Shows existing branches

Merging branches: `git merge`

- Fast-forward merge

```
Before merge:
main:    A---B
          \
feature:  C---D
```



```
After fast-forward merge:
main:    A---B---C---D
feature:          C---D
```

- Three-way merge

```
Before merge:
main:    A---B---E---F
          \
feature:  C---D
```



```
After three-way merge:
main:    A---B---E---F---M
          \
feature:  C---D---/

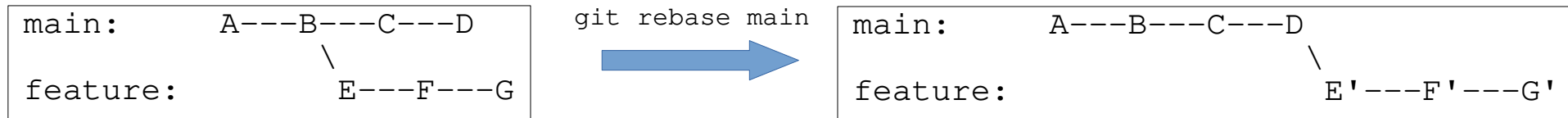
Where M is the new merge commit
```

- Syntax:

- `git merge <branch_to_merge_from>`

Merging branches: `git rebase`

- Sometimes the master branch is ahead of feature
 - You could merge the master branch into feature branch
 - Or... You may want to keep a clean linear history



- Can also be used to “squash” smaller commits!

Merge conflicts

- Same files on different branches are inconsistent

```
eric@fedora:~/branch_practice$ git merge feature
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

- Handled manually

```
This is the first line
Another line!
<<<<<< HEAD
Hello
=====
World
>>>>>> feature
```

Practice 3

1. In “reset_practice”, do a hard reset back to the commit with message “Add Feature B”.
2. Checkout the commit where you just added Feature A.
3. Branch off from this point, name your new branch “feature_c”
4. Add a new feature to feature.py:

```
def feature_c():  
    return "Feature C"
```

5. Modify main.py to read “print(“Version 4”)”
6. Merge the feature_c branch into the main/master branch. What happens?
7. Look at your git history graphically, you should see a “loop”.
8. Delete the feature_c branch.

GitHub

- Remote storage of repos
- Allows collaboration on the same project
 - Work on the same codebase simultaneously
 - Share your changes with each other
 - Keep track of who changed what and when
 - Avoid overwriting each other's work
 - Maintain a working version of the project at all times



An aside: Secure Shell (SSH)

- Cryptographic protocol that creates a secure, encrypted connection between your computer and a remote server
- Authentication & message sending
- Private and public keys



Look up “RSA cryptography” and “Diffie-Hellman key exchange” if you are interested in the underlying details

Connecting to GitHub with SSH

- Create an account on GitHub if you don't have one already
- Generate an SSH key:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

- Use ssh-agent for managing SSH access:

```
# Start SSH agent  
eval "$(ssh-agent -s)"  
  
# Add your SSH key to the agent  
ssh-add ~/.ssh/id_ed25519
```

Email you signed up
your account with



- Copy the public key and paste it to GitHub
- Test your connection with

```
ssh -T git@github.com
```


Git and GitHub

- `git clone <address to repo> [directory]`
 - Creates a “remote” – a reference to the repo you cloned from
 - Default name: origin (can be changed by `git config -global`)
 - The address should be of the form `git@github:username/repository.git`
- `git push [-u] <remote> <branch name>`
 - Lets local changes be reflected in the remote repo
 - Needs to have write-access to the repo, and have SSH set up properly
 - The -u flag makes it so that the upstream repo is remembered, future pushes are just `git push <branch name>`

Git and GitHub

- `git fetch <remote> <branch>`
 - Fetches all changes on the remote, but does not merge (safer)
 - Review changes with `git log HEAD..origin/main`
(changes in origin/main but not in local repo)
- `git pull [--[no]-rebase] <remote> <branch>`
 - Fetches all changes on the remote AND merge them with branch.
 - To make sure you don't fall behind by too much
 - In case of conflicts, can choose to either merge, or rebase

Practice 4

1. Create a repo on your GitHub account
2. Clone this repo to your local machine
3. Create a new branch, make some changes, and commit
4. Push your local branch to the remote, confirm that this branch is now on GitHub
5. Make some changes to your remote repo on GitHub
6. Fetch the changes to your local repo and use `git log` to compare
7. Merge the remote into your local branch

Pushing a local repo to GitHub

1. Create an empty repo on GitHub with the same name as your local repo
2. Copy the SSH address
3. Connect to the repo by

```
git remote add <name of remote> <URL>
```

4. Push your local repo to this new remote

Joining a collaboration

- Typical workflow:
 - Fork a repo
 - Clone the forked repo onto local machine
 - Make changes and commit
 - Push to forked repo
 - Submit a pull request (GitHub specific!)
 - People involved makes reviews and comments
 - After addressing the new changes, can merge and close the pull request

Practice 5

1. Fork the repo from https://github.com/ycsan-ust/MSDM5001_Lecture2
2. Clone the forked repo to your local machine
3. In “student_names.txt”, type in your name on a new line
4. Create a txt file titled with your date of birth: “dd_mm_yy.txt”
5. Commit changes
6. Push changes to your forked repo
7. Submit a pull request to me